Davante

12

ENTORNOS DE DESARROLLO

Diagramas de estados

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Diagramas de estados UML 4

/ 3. Estados y transiciones 4

/ 4. Caso práctico 1: “Gestión cobro impresora” 5

/ 5. Uniones, bifurcaciones y eventos 6

/ 6. Elaboración de diagramas de estados 7

/ 7. Caso práctico 2: “Centralita telefónica” 8

/ 8. Resumen y resolución del caso práctico de la unidad 8

/ 9. Bibliografía 9

© MEDAC 978-84-18983-00-9
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Comprender el funcionamiento de los diagramas de estados.

Conocer los elementos de los diagramas de estados.

Saber construir diagramas de estados.

Saber las principales características de los diagramas de estados.

/ 1. Introducción y contextualización práctica

Ya sabemos que a la hora de diseñar un proyecto de desarrollo es necesario tener en cuenta qué sucede a lo largo del tiempo de vida de ejecución de los diferentes elementos que participan en él.

Existen diferentes tipos de diagramas para poder mostrar este funcionamiento, por ejemplo, los diagramas de secuencia, que ofrecen mucha información del flujo de ejecución y de las funcionalidades de los elementos. Sin embargo, existe otro aspecto que hay que considerar, como son los cambios de estado de los objetos que participan en la ejecución, sin tener en cuenta las diferentes funcionalidades que puede tener un objeto.

Para ello, se utiliza un tipo especial de diagrama de comportamiento conocidos como diagrama de estados.

Gracias a estos diagramas, podemos crear una representación dinámica de la ejecución de la aplicación sin entrar en grandes detalles de funcionamiento.

Como veremos en este tema, el diagrama de estados está basado en fundamentos matemáticos y se pueden aplicar no solo en el diseño *software*, sino también en otros campos que no tienen que estar relacionados con el mundo informático.

Escucha el siguiente audio en el que planteamos el caso práctico que iremos resolviendo a lo largo de esta unidad.



Fig. 1. Es importante visualizar correctamente el sistema.

Audio intro. "Diagrama de estados para una prueba de usuario"
https://bit.ly/3kY7Zfl

/ 2. Diagramas de estados UML

Los **diagramas de máquinas de estados o diagramas de estados** provienen de un concepto matemático que tienen que ver con los **Autómatas Finitos Deterministas (AFD).**

Un AFD se puede utilizar en diferentes contextos para múltiples tareas, por ejemplo, **es muy utilizado en la construcción de compiladores, especialmente en la fase de análisis léxico.** Además, **estos autómatas permiten modelar y reconocer expresiones regulares.**

Desde el punto de vista de UML, los diagramas de estados se utilizan para representar condiciones de sistema o partes de este que tengan una representación finita en su comportamiento. De esta forma, estos diagramas **se usan para modelar el comportamiento dinámico de una clase o de un conjunto de ellas en respuesta a un estímulo o un evento externo.** Podríamos decir que cada clase puede tener un estado, sin embargo, no realizamos un diagrama de estados por cada clase, sino por un conjunto de ellas que representen una funcionalidad.

Una de las preguntas que típicamente nos hacemos es cuándo utilizar los diagramas de estados. Normalmente, hacemos uso de este tipo de diagramas para responder a algún tipo de evento o procesamiento. Además, podemos utilizarlo para poder modelar el comportamiento dinámico del sistema y para comprender cómo reaccionan los objetos o las clases ante un evento externo o interno.

En ocasiones, podemos confundir los diagramas de estados con los diagramas de flujo, sin embargo, existen algunas diferencias. El **objetivo** fundamental de un diagrama de estados es **mostrar diferentes cambios de los estados de las clases** y no el proceso de determinadas llamadas que causan determinados cambios. De esta forma, los diagramas de flujo permiten mostrar **cómo se procesan las órdenes y cómo se llevan acabo,** afectando el estado de los objetos.



Fig. 2. Ten en cuenta el UML para poder mejorar la calidad del software.

Audio 1. "Estados y transiciones"
https://bit.ly/2EkE4xz

/ 3. Estados y transiciones

Como su nombre indica, los diagramas de estados pueden estar formados por estados que permiten representar la situación actual de un elemento.

Desde el punto de vista de UML, en un diagrama de estados, estos se representan con un rectángulo con sus esquinas redondeadas. Así, **un estado puede representar condiciones y circunstancias de un objeto perteneciente a una clase en un momento determinado del tiempo.** Nótese que un estado es algo volátil en el tiempo y no tiene porque perdurar.

Existen dos tipos de **estados** especiales en un diagrama de estados:

- El estado **inicial,** que representa el estado en el que tiene que comenzar el diagrama.
- El estado **final,** que representa el estado en el que finaliza la secuencia.

Los diagramas de estados pueden tener más de un estado final, pero solo pueden comenzar un punto de inicio determinado.

Debemos tener en cuenta que este tipo de diagramas, al igual que sucede con otros, tienen que ser deterministas. Esto se traduce en que **ante las mismas circunstancias,** es decir, la misma entrada en el mismo estado previo, **la transición tiene que ir al mismo estado.**

Podemos decir que otro elemento de los diagramas de estados son las **transiciones**, pues estas permiten pasar de un estado a otro ante una circunstancia externa.

Normalmente, esto queda representado mediante unas líneas que van de un estado a otro con una flecha, indicando el sentido de la transición. Puede haber diferentes tipos de transiciones, por ejemplo, puede haber transiciones que nos obligan a ir a otro estado o transiciones que indiquen que tenemos que volver al mismo estado, lo que se denominan **bucles.**

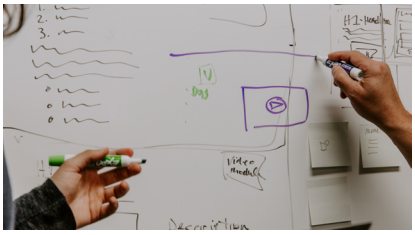


Fig. 3. Ten en cuenta las transiciones entre estados para mantener el determinismo.

Vídeo 1. "Diagrama de estados para un préstamo"
https://bit.ly/35gxd0i

/ 4. Caso práctico 1: "Gestión cobro impresora"

Planteamiento: Como parte de la documentación que tenemos que generar para el equipo de desarrollo, es necesario que creemos un diagrama de estados para que podamos representar el flujo de ejecución que tendría el software que controla una impresora de pago por uso disponible al público.

Sabemos que es posible introducir datos de control a través de la pantalla digital de la impresora. A través de estos datos, la máquina ofrecerá al usuario la posibilidad de consultar si existen ofertas. También, se puede indicar si existe un porcentaje a la hora de realizar el cobro y, por último, entregar los documentos impresos.

Nudo: ¿Cómo realizarías este diagrama?

Desenlace: Como podemos ver en la siguiente figura, existe un único punto de entrada para poder iniciar el diagrama. Se puede observar que también disponemos de un punto de finalización, que es cuando se entregan los documentos. Además, existe un estado compuesto que es el gestor de cobros, pues en este caso existe un subdiagrama que permite especificar qué sucede a la hora de realizar un cobro en nuestro sistema.

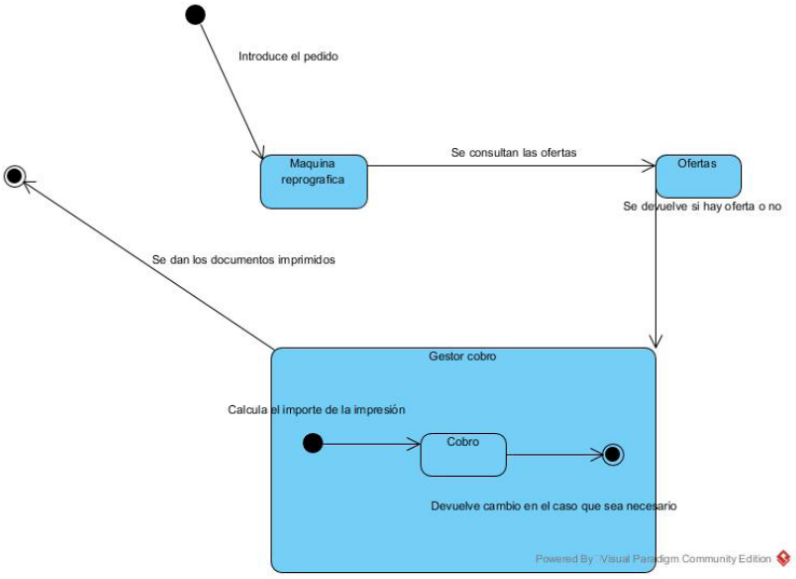


Fig. 4. Diagrama de estado para la impresora del caso.

/ 5. Uniones, bifurcaciones y eventos

A. **Uniones y bifurcaciones**

Existen otros elementos que podemos destacar en los diagramas de estados. En primer lugar, podríamos mencionar las **bifurcaciones** para representar un punto en el que podemos transitar a diferentes estados dependiendo de las condiciones de entrada.

En segundo lugar, podemos destacar las **uniones** que permiten pasar a un estado al venir de estados diferentes, es decir, es un estado de convergencia.

Por último, podemos destacar la **composición**, que se centra en representar diferentes actividades en su interior.

B. **Eventos**

En un diagrama de estado, los eventos son aquellos que permiten realizar la transición entre diferentes estados. Es cada suceso que ocurre y tiene lugar en un momento determinado del tiempo.

En los AFDs, se utilizan los eventos para poder pasar de un estado a otro. Además de esto, gracias a los eventos, los objetos y elementos pueden cambiar de estado.

Podemos distinguir diferentes tipos de eventos:

» **Síncronos.** Son aquellos que tienen que esperar para continuar el flujo de ejecución.

» **Asíncronos.** No requieren que se finalice para poder continuar el flujo de ejecución.

Por otra parte, es posible clasificar los eventos teniendo en cuenta otro tipo de parámetros, como los que mostramos a continuación:

» **Externos.** Se trata de eventos que tienen lugar en otras partes del sistema que no están representadas con el diagrama de estados actual.

» **Internos.** Se trata de aquellos eventos que tienen lugar entre los estados que están representados en el diagrama que nos concierne.

» **Señales.** Permiten representar las acciones de una manera asíncrona para que otro estado lo pueda recibir. Normalmente, coincide con el envío de señales en el software.

» **Eventos de llamada.** Son eventos síncronos, por regla general, que permiten la representación de la ejecución de un método o funcionalidad de las operaciones de un objeto.



Vídeo 2. "Estado de funcionamiento de un editor de texto"
https:/bit.ly/2XpC4cP

/ 6. Elaboración de diagramas de estados

A la hora de dibujar los diagramas de estados, es recomendable seguir **una serie de pasos:**

- Identificar el estado inicial y los estados de finalización.
- Identificar los diferentes estados y los objetos que pueden existir en ellos, uniéndolos con los correspondientes atributos.
- Etiquetar los eventos que puedan servir para lanzar la transición.

Cuando creamos el diagrama de estados, va a depender del sistema que queremos representar. Por ejemplo, no será lo mismo representar el flujo de ejecución de un sistema de reserva de vuelos, que el flujo de ejecución de un sistema para el control de acceso.

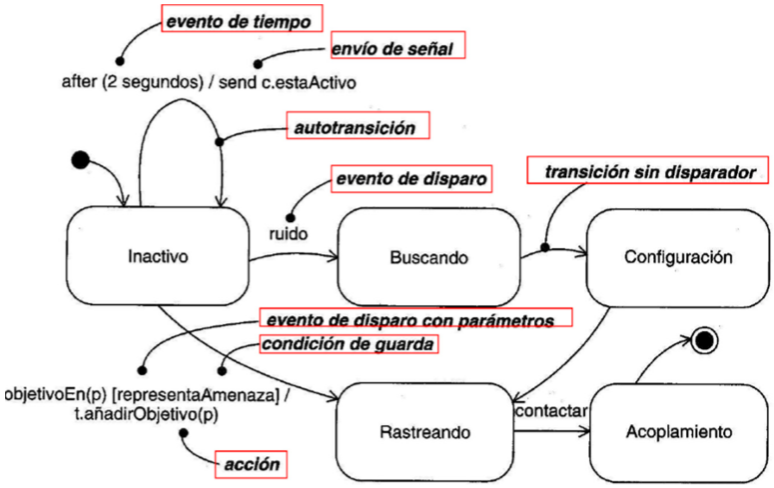


Fig. 5. Ejemplo de diagrama de estados.

Es posible que a la hora de crear un diagrama de estados existan estados con sus propios estados en su interior. Esto quiere decir que es necesario resolver el flujo de ejecución de este estado para poder saber qué sucede cuando llegamos al mismo. Normalmente, esto equivale a ampliar internamente un estado, pues es muy complicado encontrar estados que sean unitarios y no requieran ninguna otra acción.

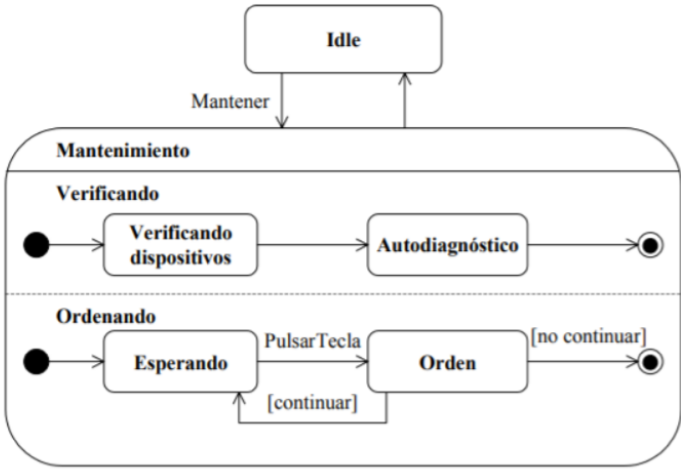


Fig. 6. Ejemplo de diagrama de estado con estados con secuencias.

A modo de resumen, los **elementos más importantes** que debemos tener en cuenta a la hora de crear un diagrama de estados **son los estados y las transiciones** que pueden implicar un cambio de estado.

/ 7. Caso práctico 2: "Centralita telefónica"

Planteamiento: Como desarrollador *software*, nos llega el siguiente diagrama de estados para implementar la funcionalidad en una centralita telefónica:

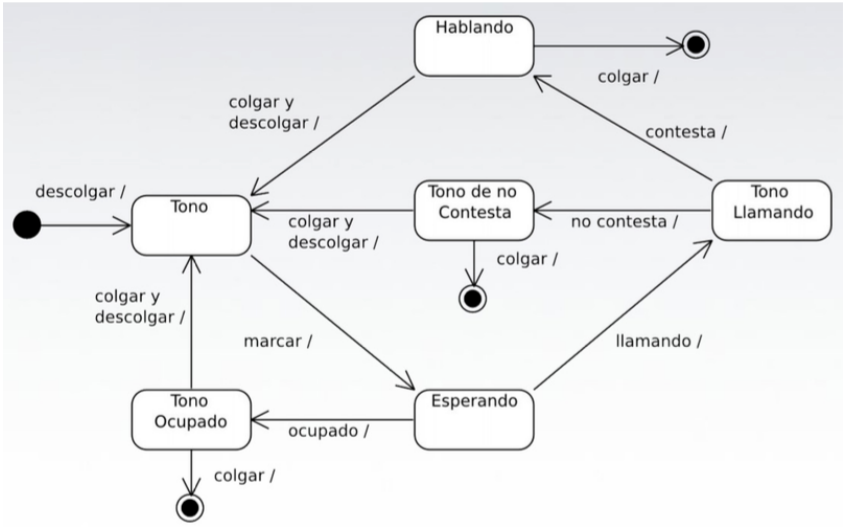


Fig. 7. Flujo de ejecución de una centralita.

Nudo: ¿Cómo interpretarías este diagrama?

Desenlace: Como podemos ver en el diagrama, existe un estado inicial que cuando recibe el evento *descolgar* transitamos al estado Tono. Estando en este estado, únicamente podemos *marcar*.

Durante el marcado, nos podemos quedar *esperando*. En este estado, pueden pasar dos cosas: que el teléfono que estemos marcando esté ocupado o que recibamos tono de *llamada*.

Si recibimos el *tono de ocupado*, podemos *colgar y finalizar* o *colgar y descolgar* para volver a coger *tono* (estado Tono).

En el caso de tener *tono de llamada*, pasemos al estado de *Tono Llamando* en el que pueden pasar dos cosas: *contesta o no contesta.*

Si las llamadas se desarrollan correctamente, quiere decir que han contestado y hemos hablado (estado *Hablando*) y podemos colgar (finalizando) o volver al principio colgando y volviendo a llamar.

Finalmente, si no contestan, podemos *colgar y finalizar*, o bien *colgar* y volver a descolgar para empezar una nueva *llamada*.

/ 8. Resumen y resolución del caso práctico de la unidad

En este tema, hemos presentado los **diagramas de estados** y su aplicación dentro del modelado de aplicaciones, utilizando el lenguaje UML.

Como hemos visto, este tipo de diagramas permite mostrar los cambios de estados de los elementos que modelemos, entendiendo por estos elementos los **objetos** en ejecución.

Hemos podido observar que estos diagramas no están directamente relacionados con el mundo informático, sino que se pueden **utilizar en diferentes contextos,** especialmente en el ámbito matemático. De ahí que exista una especialización para utilizarlos con el fin de modelar el *software*.

Para la creación de este tipo de diagramas, debemos **conocer diferentes elementos** que pueden participar en su construcción, por ejemplo, los tipos de eventos que pueden ocurrir, los tipos de transiciones que podemos incluir en el diagrama, etc.

Gracias a este tipo de diagramas, podemos dar una **visión dinámica del flujo de ejecución** de los objetos indicando solo los cambios de estado sin entrar grandes detalles de implementación.

Resolución del caso práctico inicial

Como hemos visto en el tema, los diagramas de estado pueden servir para aclarar el flujo que tiene que seguir una aplicación para los desarrolladores. Además, también puede ser utilizada para explicar a otros individuos cómo debe comportarse la aplicación. Por ejemplo, pueden ser utilizados por los comerciales para explicar el flujo a nuestros clientes.

En el enunciado del caso, necesitamos explicar a los probadores o *testers* cómo debe comportarse la aplicación y los resultados esperados. Esto sería un claro ejemplo de por qué los desarrolladores tienen que saber crear diagramas de estado que permita transmitir la información cuando sea necesario.

Un posible diagrama de estados que representa la funcionalidad de inicio de sesión planteada en el caso puede ser el siguiente:

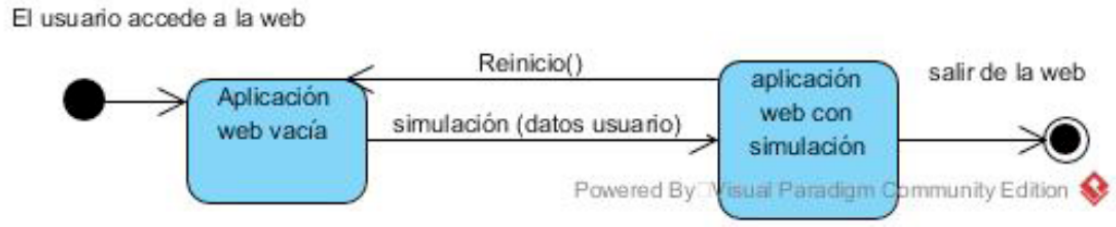


Fig. 8. Diagrama de estados para el caso planteado.

/ 9. Bibliografía

Pender, T., McSheffrey, E. & Varveris, L. (2003). *UML bible*. Indianapolis, Ind: Wiley Pub.

Juan, J. & Resusta, L. (2014). UML práctico: *aprende UML paso a paso*. España: editor no identificado.